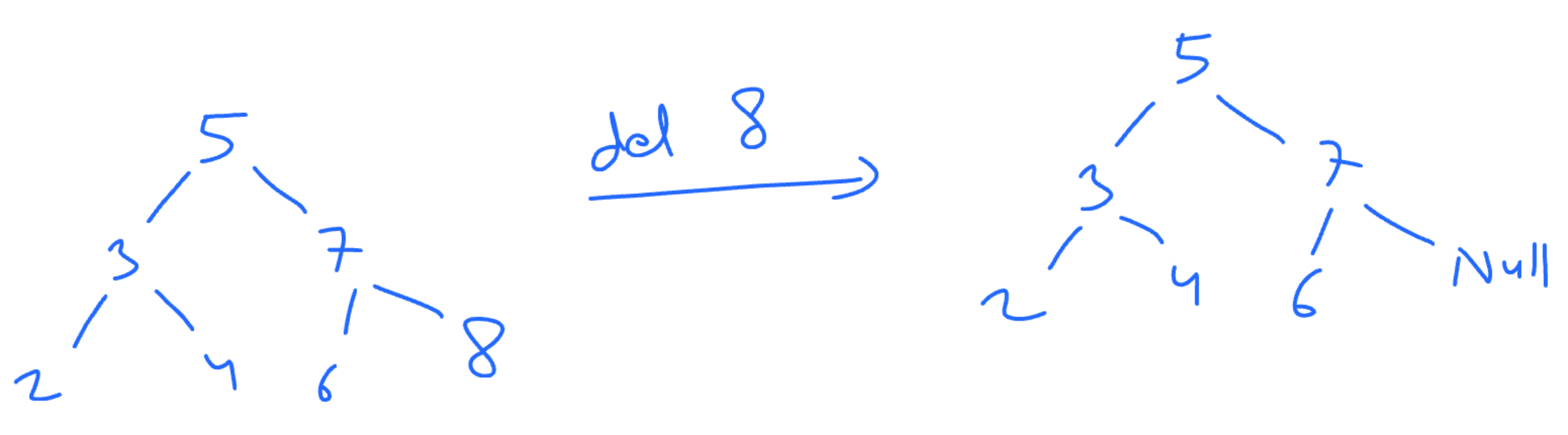
# BST.

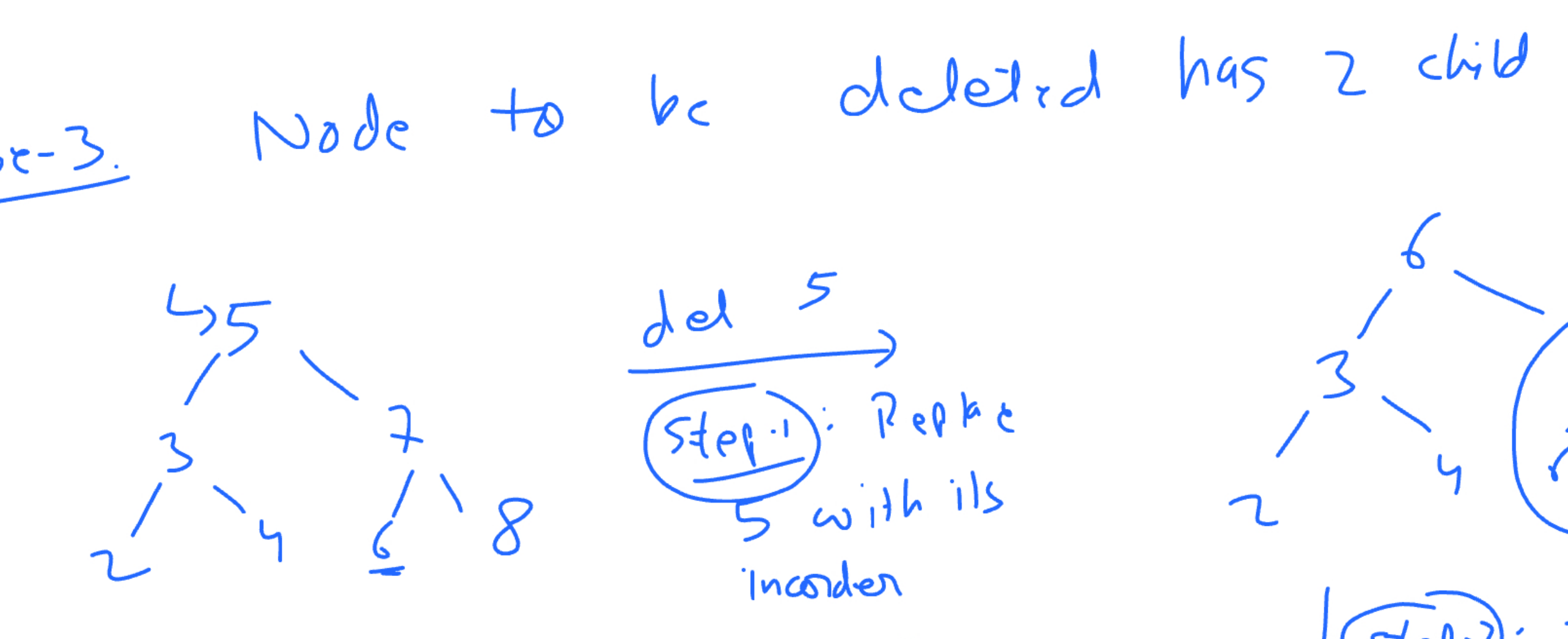
## Deletion

**Case-1** - Node to be deleted has no child nodes.



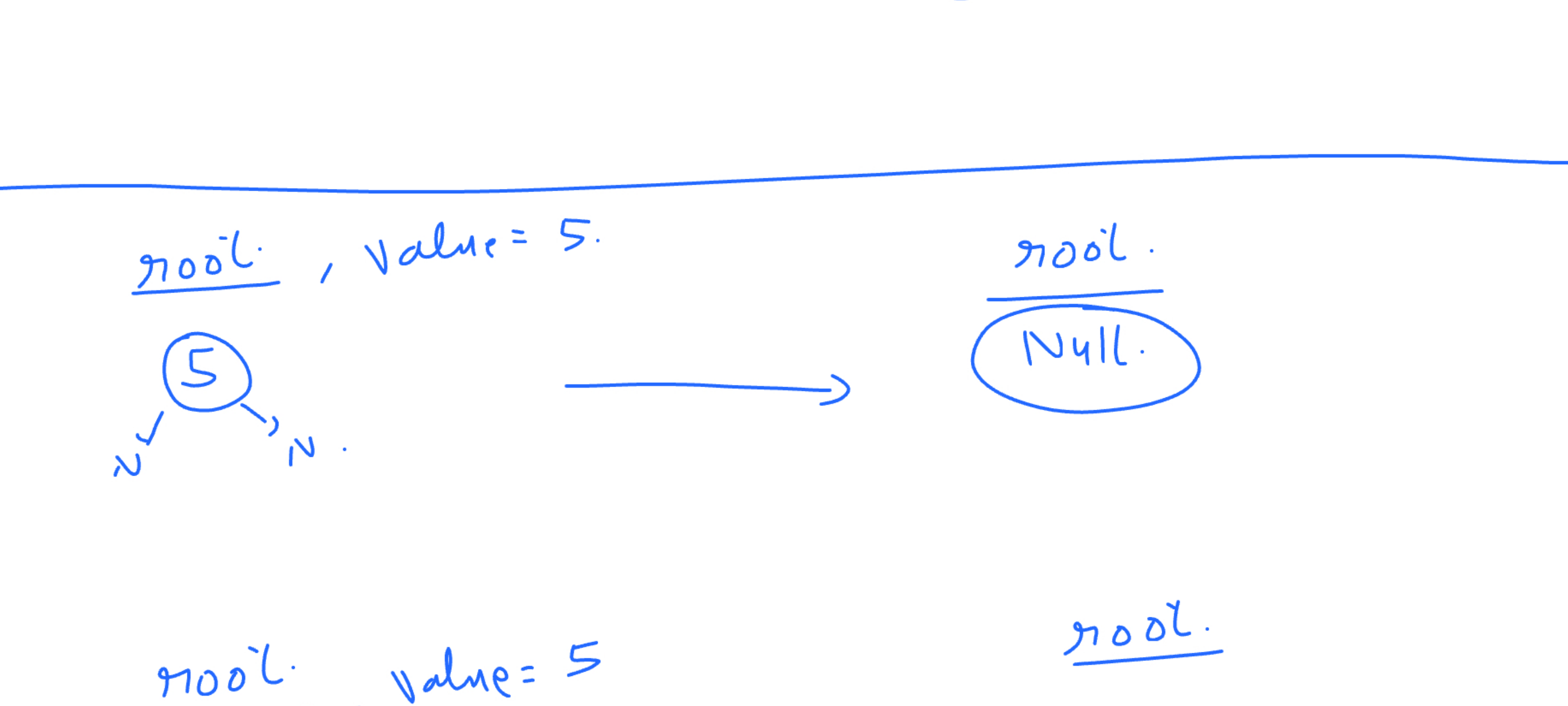
=> Replace the node with Null.

**Case-2.** Node to be deleted has 1 child node.



=> Replace the node with its non-Null child node.

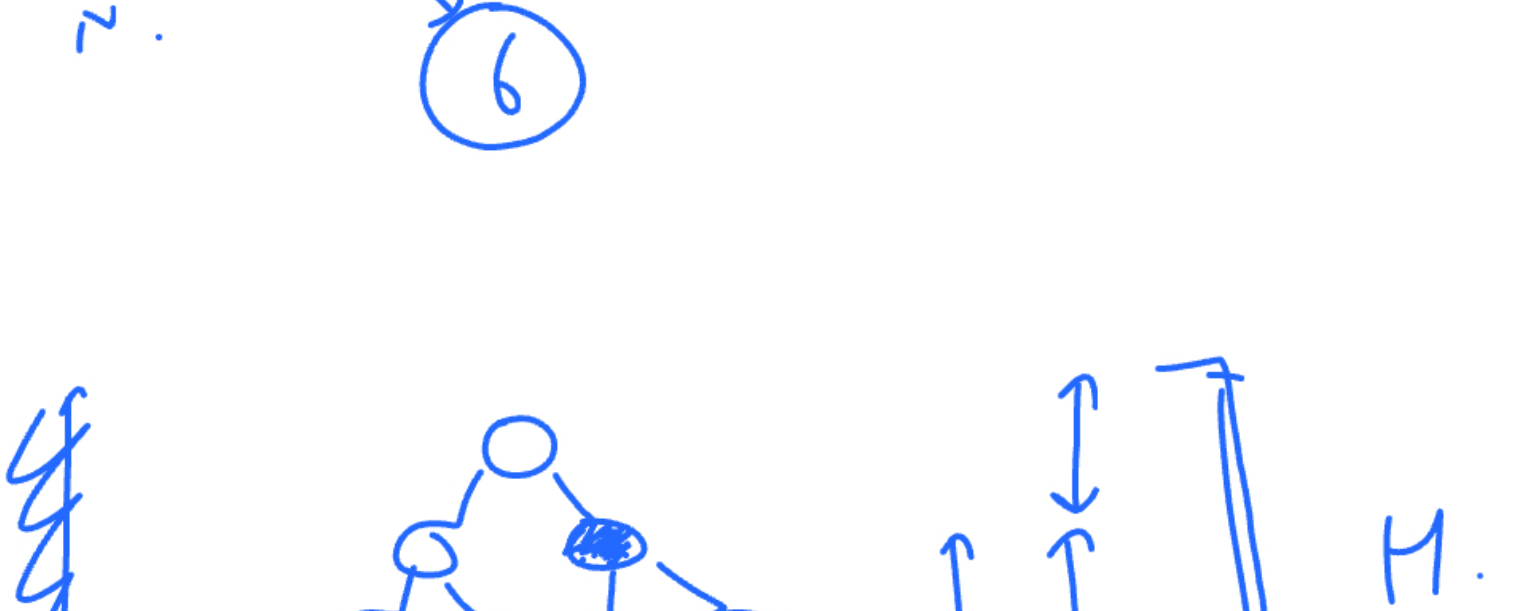
**Case-3.** Node to be deleted has 2 child nodes.

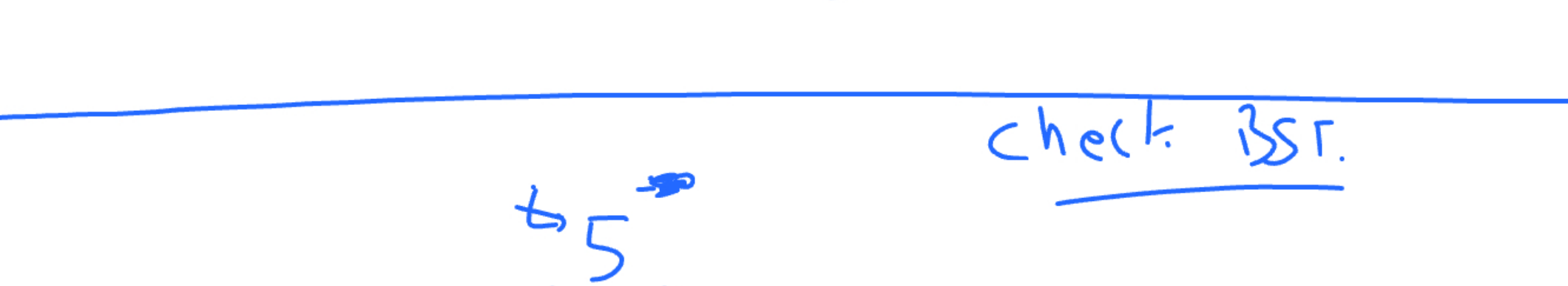


Step-1: Replace 5 with its inorder successor.

Step-2: Delete the inorder successor.

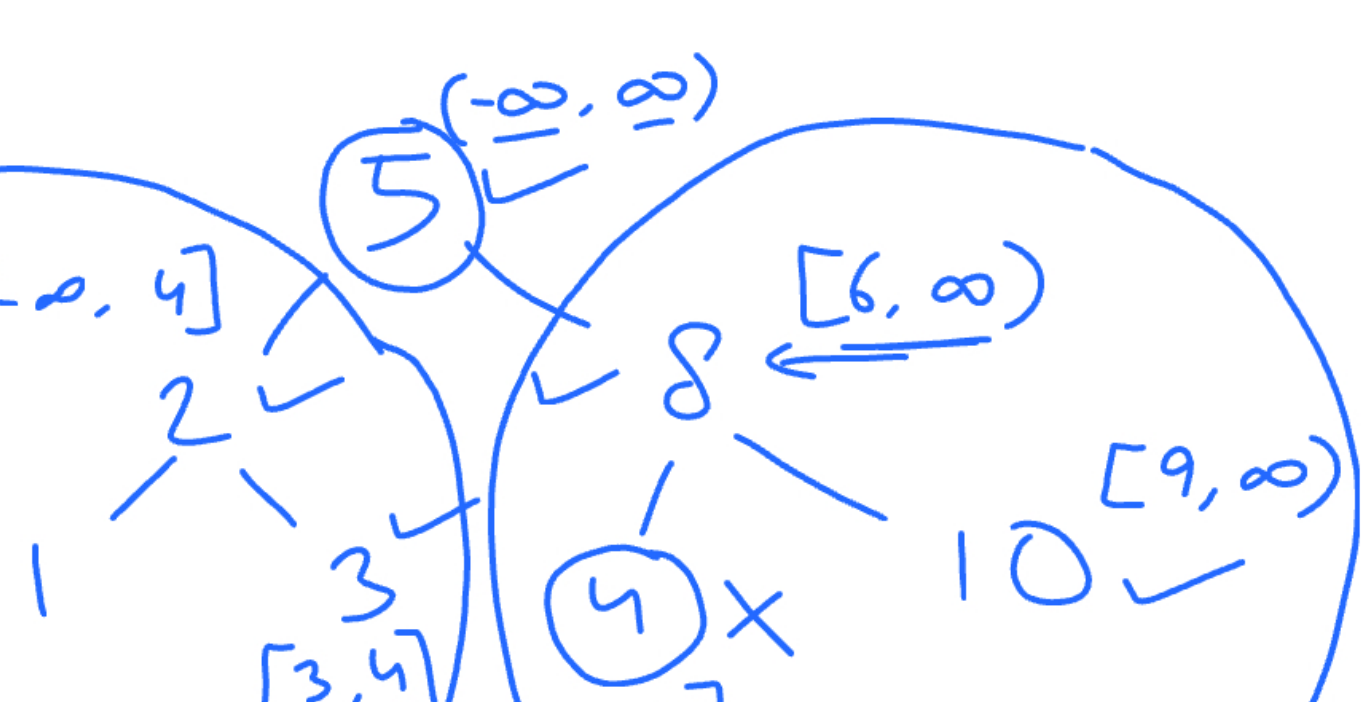
---





---

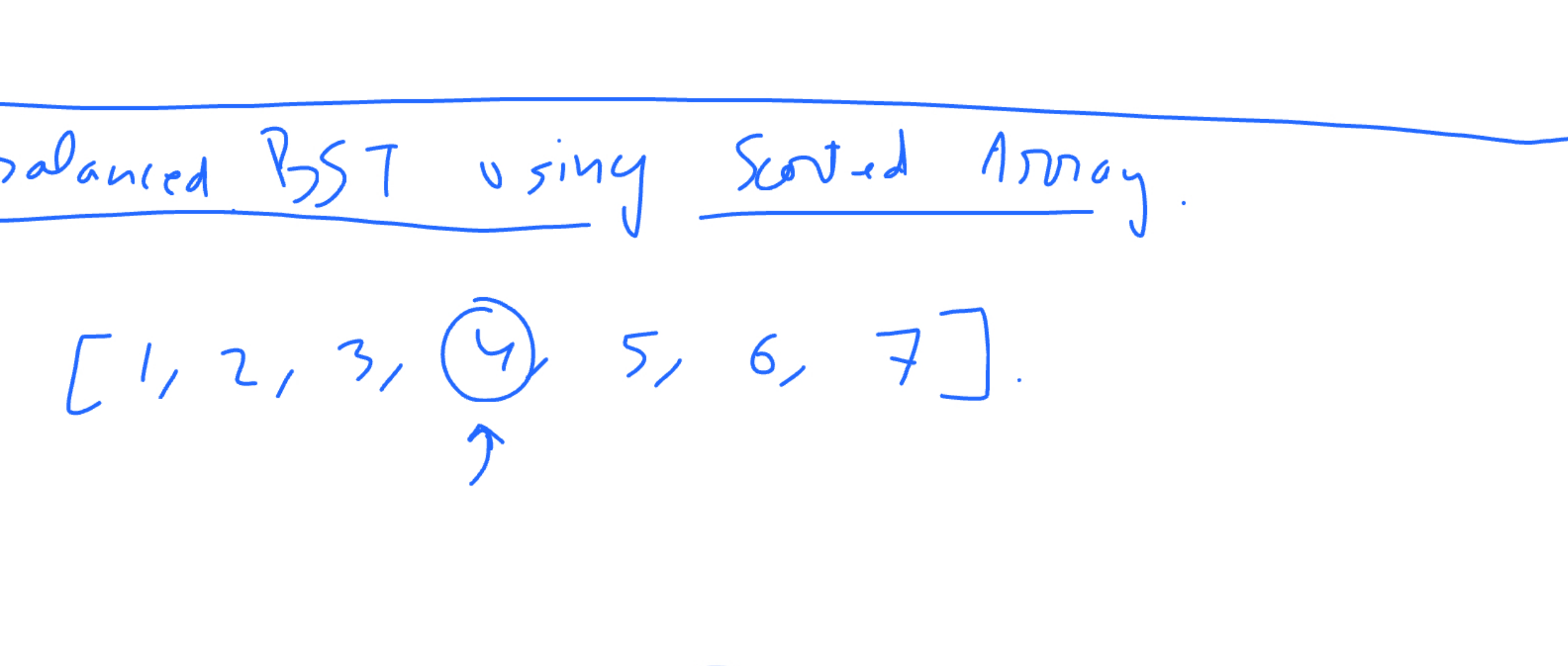
## check BST.



prev: ~~-∞~~ X
~~2~~ ~~3~~ ~~5~~
~~7~~ 8 10

TC : O(n).
AS : O(H)

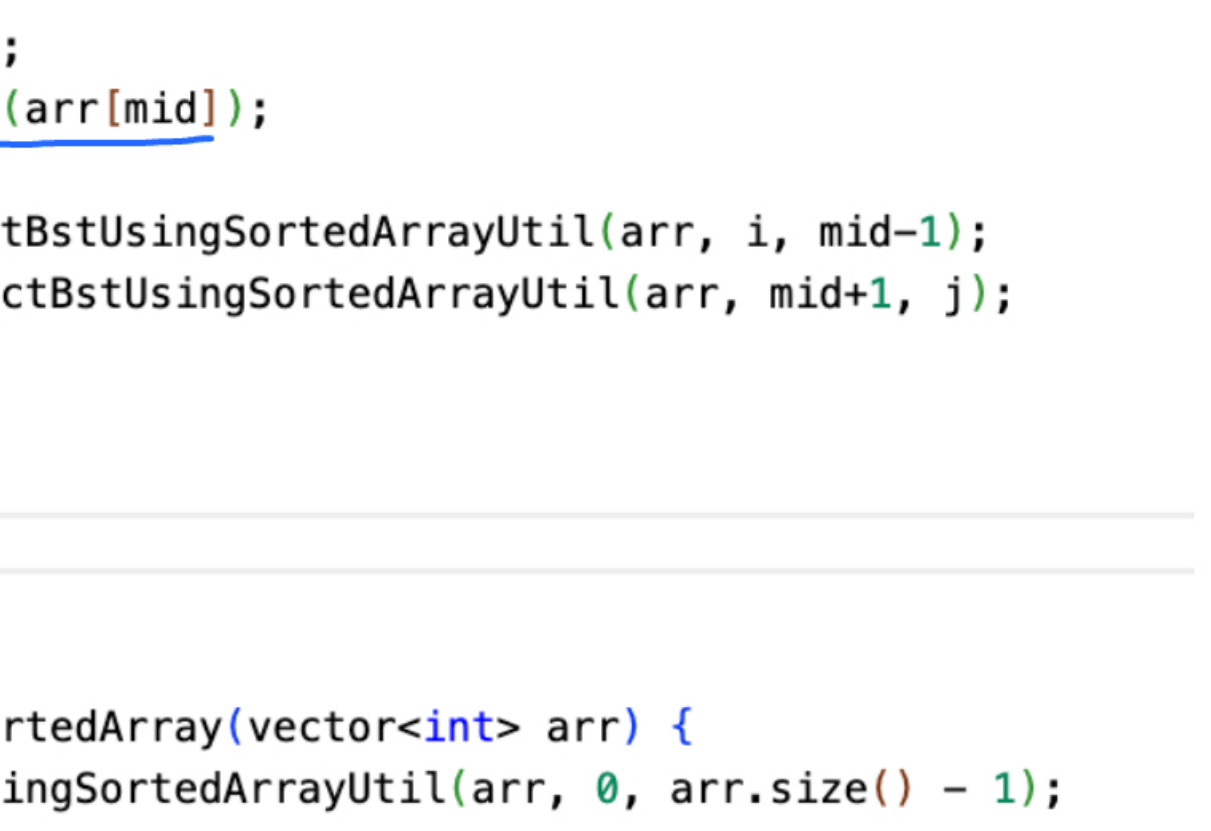
(root , prev)

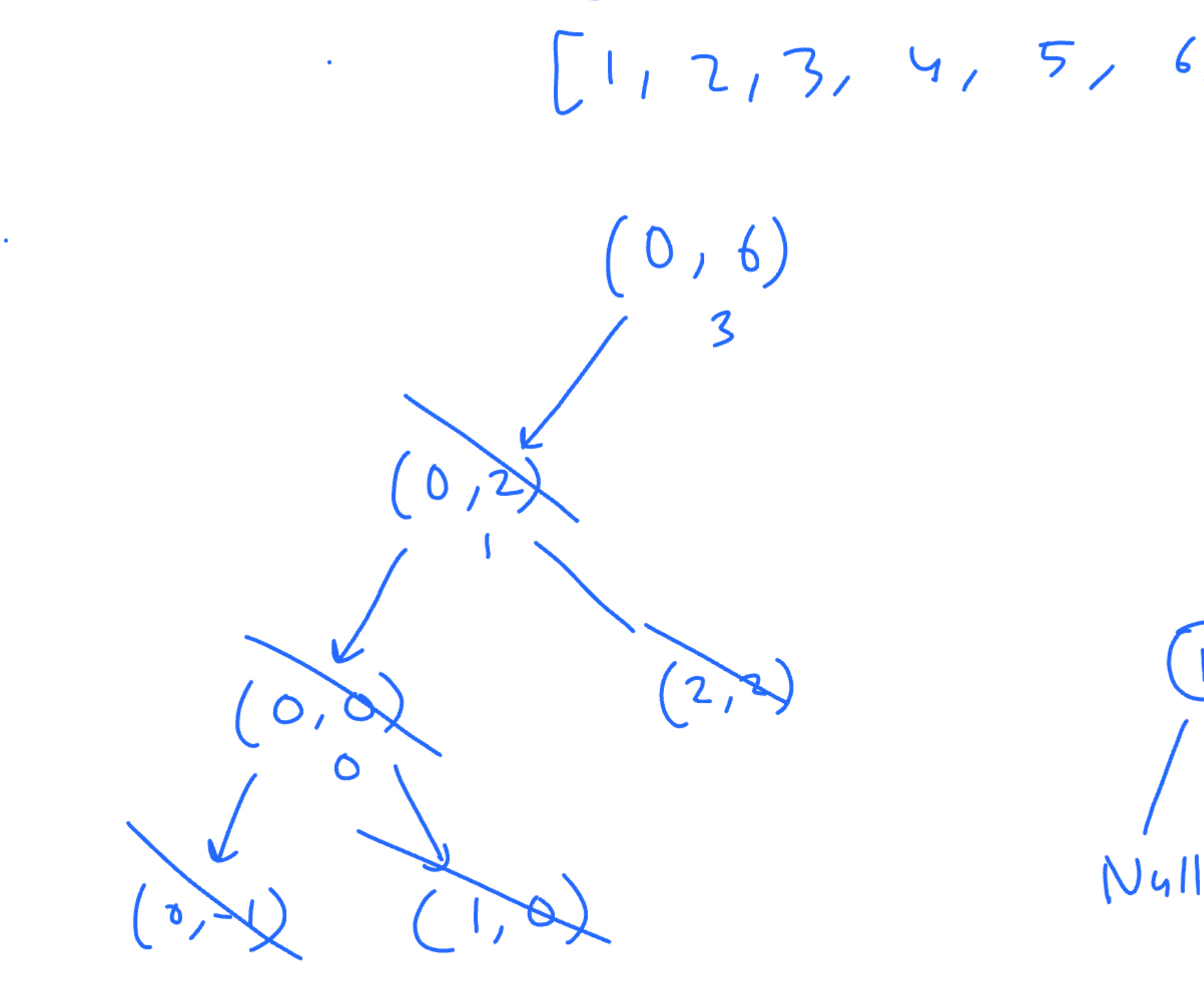


---

## Ht. Balanced BST using Sorted Array.

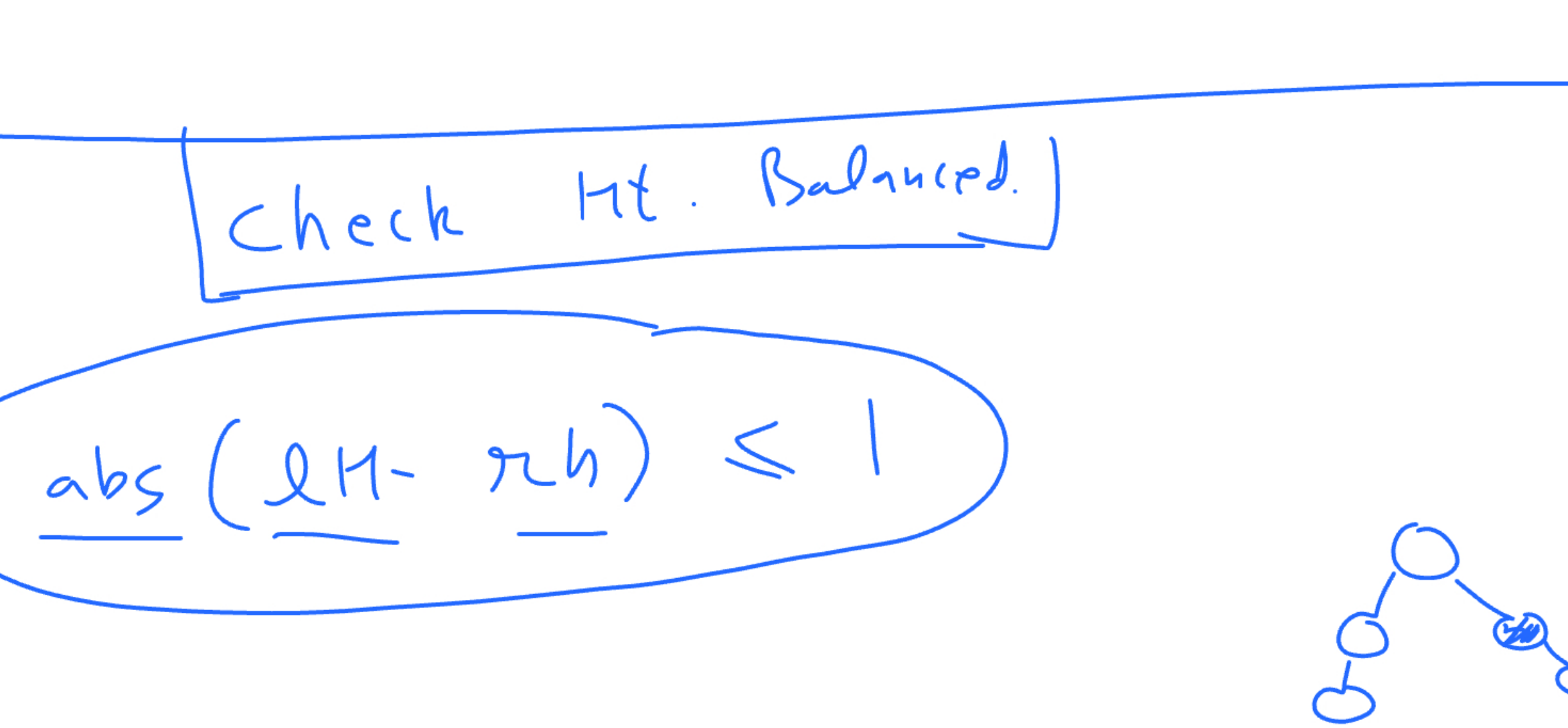
[1, 2, 3, 4, 5, 6, 7].





```
Node* constructBstUsingSortedArrayUtil(vector<int> &arr, int i, int j) {
    if (i > j) {
        return NULL;
    }

    int mid = (i + j) / 2;
    Node* root = new Node(arr[mid]);

    root->left = constructBstUsingSortedArrayUtil(arr, i, mid-1);
    root->right = constructBstUsingSortedArrayUtil(arr, mid+1, j);

    return root;
}

// TC: O(n)
// AS: O(log(n))
Node* constructBstUsingSortedArray(vector<int> arr) {
    return constructBstUsingSortedArrayUtil(arr, 0, arr.size() - 1);
}
```

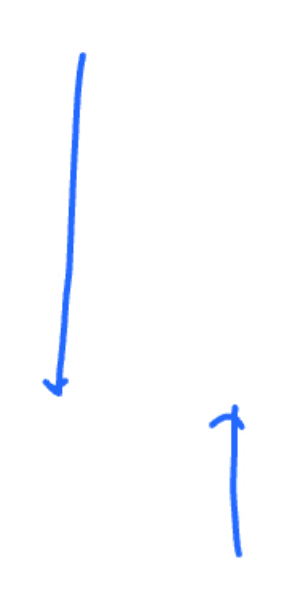
0 1 2 3 4 5 6.
[1, 2, 3, 4, 5, 6, 7].

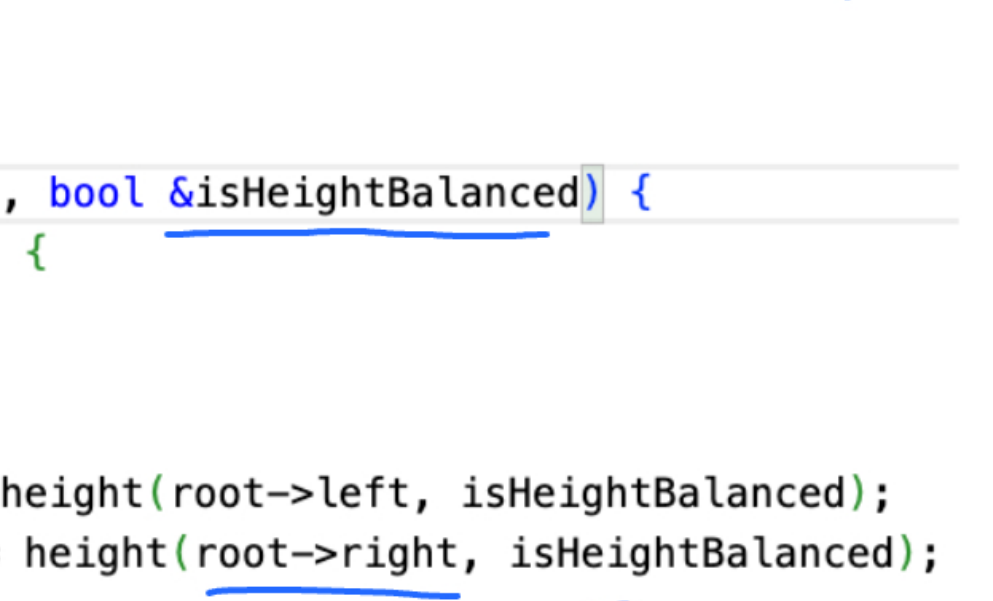


---

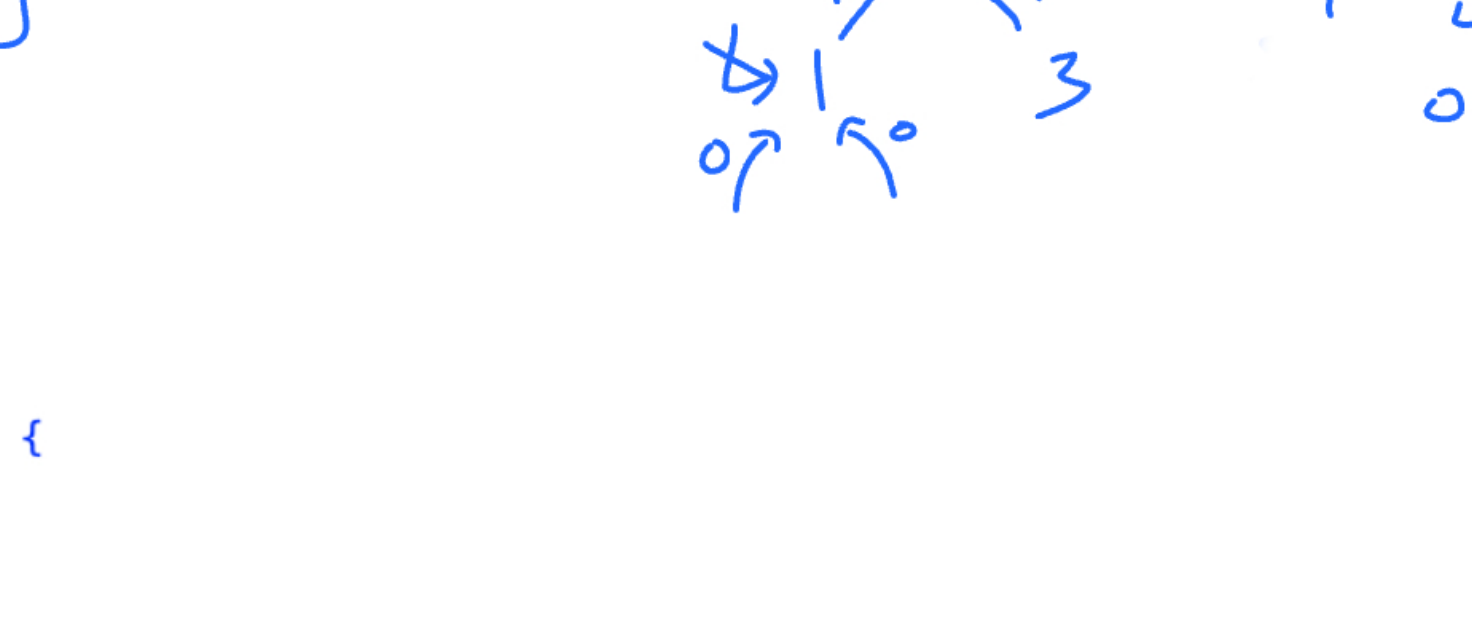
## Check Ht. Balanced.

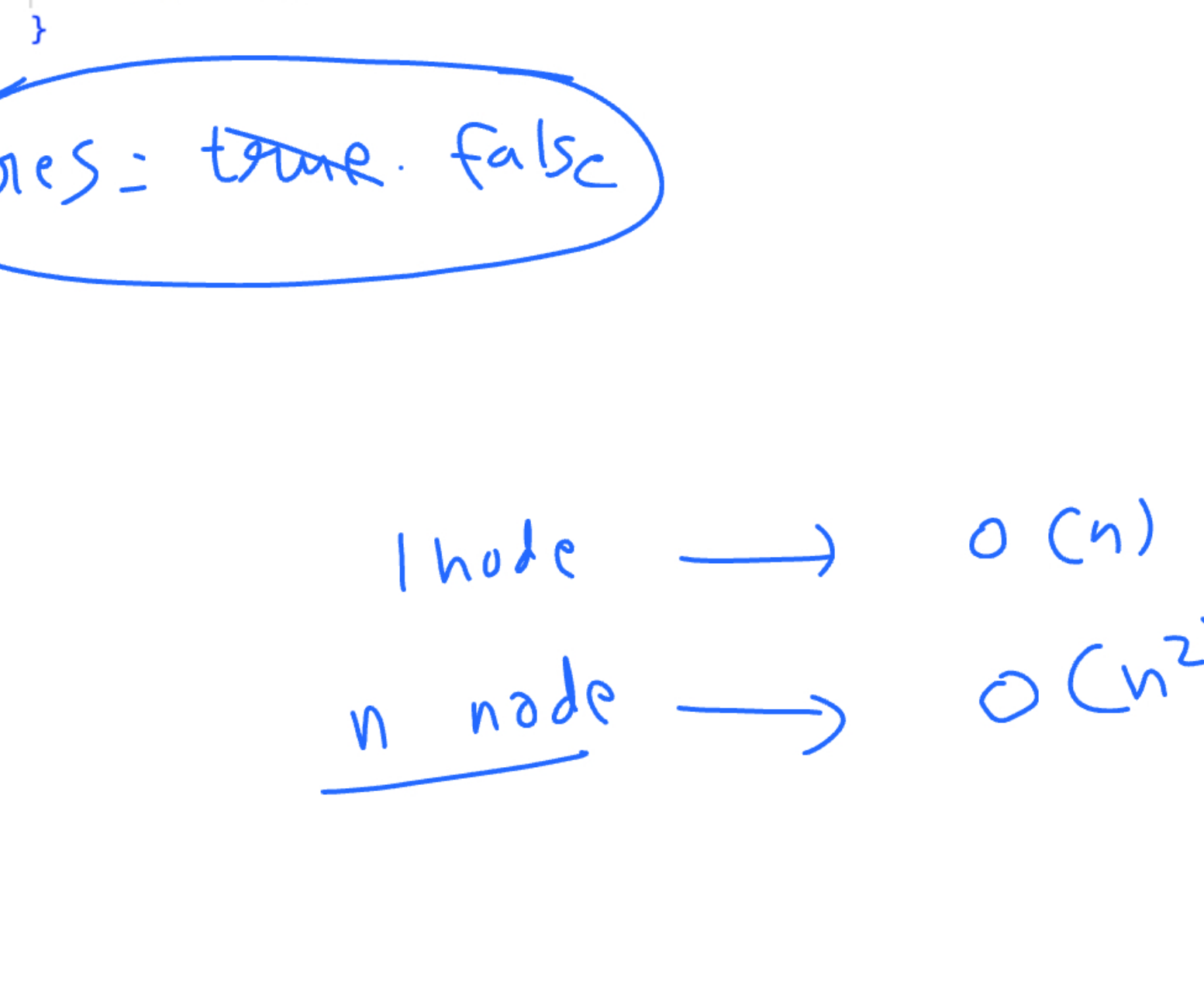
abs(lH - rh) ≤ 1



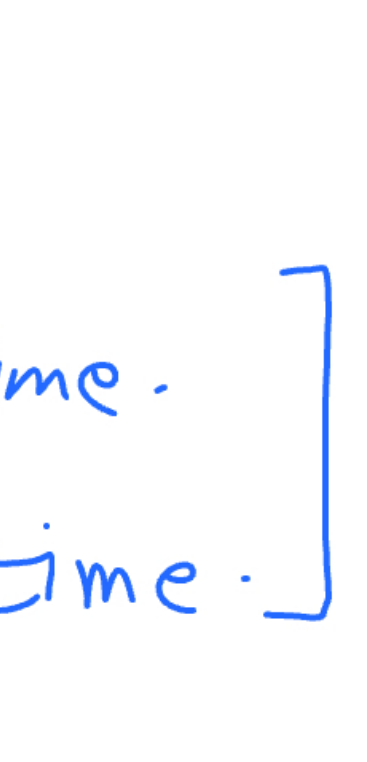


4-2-1





```
int height(Node* root, bool &isHeightBalanced) {
    if (root == NULL) {
        return 0;
    }

    int leftHeight = height(root->left, isHeightBalanced);
    int rightHeight = height(root->right, isHeightBalanced);

    if (abs(leftHeight - rightHeight) > 1) {
        isHeightBalanced = false;
    }

    return 1 + max(leftHeight, rightHeight);
}

// TC: O(n)
// AS: O(H)
bool isHeightBalancedUsingHeightFn(Node* root) {
    bool result = true;
    height(root, result);
    return result;
}
```



res: ~~true~~. false

1 node → O(n) time.
n node → O(n²) time.